

# Mamba-Enhanced GraphCast: Current Method and Setup

Lianghong Mo

April 10, 2026

## Abstract

This document describes the current implementation of temporal memory in GraphCast using a Mamba selective state space model (SSM). We explain each component from first principles: how GraphCast predicts weather, what Mamba is and how it provides temporal memory, how we combine them via a residual connection, and how we train the combined model using sequential sampling with truncated backpropagation through time (TBPTT). Every parameter is defined with its physical meaning.

## Contents

|          |                                                             |          |
|----------|-------------------------------------------------------------|----------|
| <b>1</b> | <b>GraphCast: How It Predicts Weather</b>                   | <b>2</b> |
| 1.1      | The Task . . . . .                                          | 2        |
| 1.2      | The Pipeline . . . . .                                      | 2        |
| 1.2.1    | Stage 1: Feature Construction . . . . .                     | 2        |
| 1.2.2    | Stage 2: Grid-to-Mesh Encoder . . . . .                     | 2        |
| 1.2.3    | Stage 3: Mesh Processor . . . . .                           | 3        |
| 1.2.4    | Stage 4: Mesh-to-Grid Decoder . . . . .                     | 3        |
| 1.3      | Multi-Step Forecasting (Autoregressive Rollout) . . . . .   | 3        |
| <b>2</b> | <b>Mamba: How It Provides Memory</b>                        | <b>3</b> |
| 2.1      | The Selective State Space Model . . . . .                   | 3        |
| 2.2      | The Decay Rate: How Long Is the Memory? . . . . .           | 4        |
| 2.3      | The Full SSM Block . . . . .                                | 4        |
| <b>3</b> | <b>Combining GraphCast and Mamba</b>                        | <b>4</b> |
| 3.1      | Where Mamba Is Inserted . . . . .                           | 4        |
| 3.2      | What Mamba Does at Each Prediction Step . . . . .           | 5        |
| 3.3      | State Shape and Batch Handling . . . . .                    | 5        |
| <b>4</b> | <b>Training: Sequential Sampling with Truncated BPTT</b>    | <b>6</b> |
| 4.1      | The Problem with Random Sampling . . . . .                  | 6        |
| 4.2      | Our Solution: Chunked Sequential Sampling . . . . .         | 6        |
| 4.3      | Step-by-Step Training Procedure . . . . .                   | 6        |
| 4.4      | Truncated BPTT: What Gradients Can and Cannot See . . . . . | 7        |
| <b>5</b> | <b>Inference Procedure</b>                                  | <b>7</b> |
| <b>6</b> | <b>Complete Parameter Table</b>                             | <b>8</b> |
| <b>7</b> | <b>Preliminary Results</b>                                  | <b>9</b> |

# 1 GraphCast: How It Predicts Weather

## 1.1 The Task

Given the atmospheric state at two consecutive times ( $t-6h$  and  $t$ ), predict the state at  $t+6h$ . The atmospheric state consists of:

- **Atmospheric variables** (at each pressure level): temperature, geopotential, u/v wind components, vertical velocity, specific humidity
- **Surface variables**: 2m temperature, 10m u/v wind, mean sea level pressure, 6h precipitation
- **Forcing variables** (known in advance): solar radiation, time-of-year, time-of-day
- **Static variables**: surface geopotential, land-sea mask

These are defined on a regular latitude-longitude grid. At resolution  $2.0^\circ$ , this is a  $91 \times 180$  grid (16,380 points). At each grid point, there are 6 atmospheric variables  $\times$  13 pressure levels + 5 surface variables = 83 scalar values per time step.

## 1.2 The Pipeline

GraphCast [1] processes one prediction step through four stages:

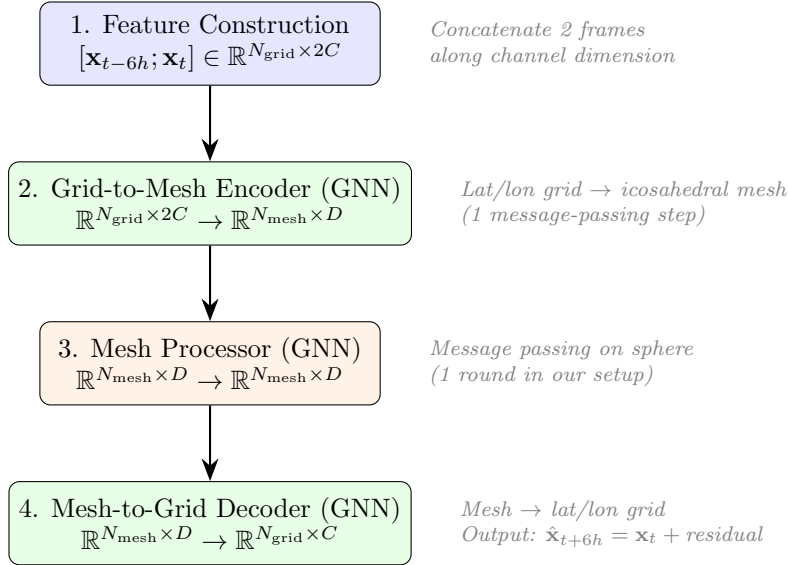


Figure 1: GraphCast pipeline for a single 6-hour prediction step.

### 1.2.1 Stage 1: Feature Construction

The two input frames are **concatenated along the channel dimension**:

$$\mathbf{f}_i = [\mathbf{x}_{t-6h}^{(i)}; \mathbf{x}_t^{(i)}] \in \mathbb{R}^{2C}$$

where  $i$  indexes the  $N_{\text{grid}}$  grid points and  $C$  is the number of variables per time step. This allows downstream MLPs to learn temporal features like velocity ( $\mathbf{x}_t - \mathbf{x}_{t-6h}$ ) from the concatenated input.

### 1.2.2 Stage 2: Grid-to-Mesh Encoder

A graph neural network maps features from the regular lat/lon grid to an **icosahedral mesh**—a triangulation of the sphere with approximately uniform node spacing. With `mesh_size=4`, this mesh has  $N_{\text{mesh}} = 2,562$  nodes.

The encoder performs one message-passing step on a bipartite graph (grid nodes  $\rightarrow$  mesh nodes), producing a mesh latent:

$$\mathbf{m} \in \mathbb{R}^{N_{\text{mesh}} \times D}$$

where  $D = 128$  is the latent feature dimension (**width**).

### 1.2.3 Stage 3: Mesh Processor

The processor runs message passing on the mesh graph. Each mesh node aggregates information from its mesh neighbors:

$$\mathbf{m}_j^{(k+1)} = \mathbf{m}_j^{(k)} + \text{MLP}\left(\mathbf{m}_j^{(k)}, \sum_{j'} \text{MLP}([\mathbf{m}_j^{(k)}; \mathbf{m}_{j'}^{(k)}; \mathbf{e}_{jj'}])\right)$$

In our setup, this runs for 1 round (`gnn_msg.steps=1`).

### 1.2.4 Stage 4: Mesh-to-Grid Decoder

A reverse GNN maps mesh features back to the grid. The output is a **residual** prediction:

$$\hat{\mathbf{x}}_{t+6h} = \mathbf{x}_t + \text{decode}(\mathbf{m}^{(\text{final})})$$

## 1.3 Multi-Step Forecasting (Autoregressive Rollout)

For forecasts beyond 6 hours, the prediction is fed back as input:

$$\underbrace{[\mathbf{x}_{t-6h}, \mathbf{x}_t]}_{\text{input}} \xrightarrow{\text{predict}} \hat{\mathbf{x}}_{t+6h} \xrightarrow{\text{feed back}} \underbrace{[\mathbf{x}_t, \hat{\mathbf{x}}_{t+6h}]}_{\text{new input}} \xrightarrow{\text{predict}} \hat{\mathbf{x}}_{t+12h} \rightarrow \dots$$

**The limitation:** Each prediction step sees only the 2 most recent frames. At step  $k$ , the model has no knowledge of the original initial conditions or any atmospheric state before step  $k-1$ . This is the problem we address with Mamba.

## 2 Mamba: How It Provides Memory

### 2.1 The Selective State Space Model

Mamba [2] is a sequence model that maintains a hidden state  $h \in \mathbb{R}^H$  which accumulates information over time:

$$h_t = \underbrace{\bar{A}_t}_{\text{decay}} \cdot h_{t-1} + \underbrace{(1 - \bar{A}_t)}_{\text{update}} \cdot u_t \quad (1)$$

where:

- $h_t \in \mathbb{R}^H$  is the hidden state at time  $t$  (the “memory”)
- $u_t \in \mathbb{R}^H$  is the projected input at time  $t$
- $\bar{A}_t = \exp(A \cdot \Delta_t)$  is the **data-dependent decay** (how much to forget)
- $A \in \mathbb{R}^H$  is a learnable log-decay parameter (initialized to  $-0.1$ )
- $\Delta_t = \text{softplus}(\text{Linear}(u_t))$  is the **data-dependent step size**

**Intuition:** At each time step, the hidden state is a weighted average of the old state and the new input. When  $\bar{A}_t \approx 1$  (small  $\Delta_t$ ), the state is mostly preserved (“remember”). When  $\bar{A}_t \approx 0$  (large  $\Delta_t$ ), the state is mostly replaced (“forget and update”).

## 2.2 The Decay Rate: How Long Is the Memory?

The parameter `a_log` (initialized to  $-0.1$ ) controls the default decay:

$$A = -\exp(\text{a\_log}) = -\exp(-0.1) \approx -0.905$$

The decay per step is  $\bar{A} = \exp(A \cdot \Delta)$ . For a typical  $\Delta \approx 1$ :

$$\bar{A} \approx \exp(-0.905) \approx 0.40$$

After  $k$  steps without update, the state decays to  $\bar{A}^k$ :

| Steps     | 1   | 3    | 5    | 10    | 20         |
|-----------|-----|------|------|-------|------------|
| Time      | 6h  | 18h  | 30h  | 2.5d  | 5d         |
| Remaining | 40% | 6.5% | 1.1% | 0.01% | $\sim 0\%$ |

In practice,  $\Delta_t$  is data-dependent and learned, so the effective memory length varies. But the initialization gives the model a starting point of roughly 1–3 steps (6–18 hours) of strong memory.

## 2.3 The Full SSM Block

Our Mamba block takes a mesh latent  $\mathbf{m} \in \mathbb{R}^{N_{\text{mesh}} \times D}$  and produces an output of the same shape:

1. **Layer normalization:**  $\tilde{\mathbf{m}} = \text{LayerNorm}(\mathbf{m})$
2. **Input projection:**  $[\mathbf{u}, \mathbf{g}] = \text{Linear}_{2H}(\tilde{\mathbf{m}})$  (split into value  $\mathbf{u}$  and gate  $\mathbf{g}$ , each  $\in \mathbb{R}^H$ )
3. **Step size:**  $\Delta = \text{softplus}(\text{Linear}_H(\mathbf{u}))$  (data-dependent discretization)
4. **State update:**  $h_{\text{new}} = \exp(A \cdot \Delta) \cdot h_{\text{prev}} + (1 - \exp(A \cdot \Delta)) \cdot \mathbf{u}$
5. **Gated output:**  $\mathbf{y} = h_{\text{new}} \odot \sigma(\mathbf{g}) + \text{skip} \cdot \mathbf{u}$  (sigmoid gating + skip connection)
6. **Output projection:**  $\Delta \mathbf{m} = \text{Linear}_D(\mathbf{y})$  (project back to mesh latent dimension)
7. **Residual:**  $\mathbf{m}' = \mathbf{m} + \Delta \mathbf{m}$  (add correction to original)

The residual connection in step 7 is critical: if Mamba learns nothing useful,  $\Delta \mathbf{m} \rightarrow \mathbf{0}$  and the output equals the input (baseline behavior).

# 3 Combining GraphCast and Mamba

## 3.1 Where Mamba Is Inserted

We insert the Mamba block **between the encoder and processor** (location: `mesh_post_encoder_residual`):

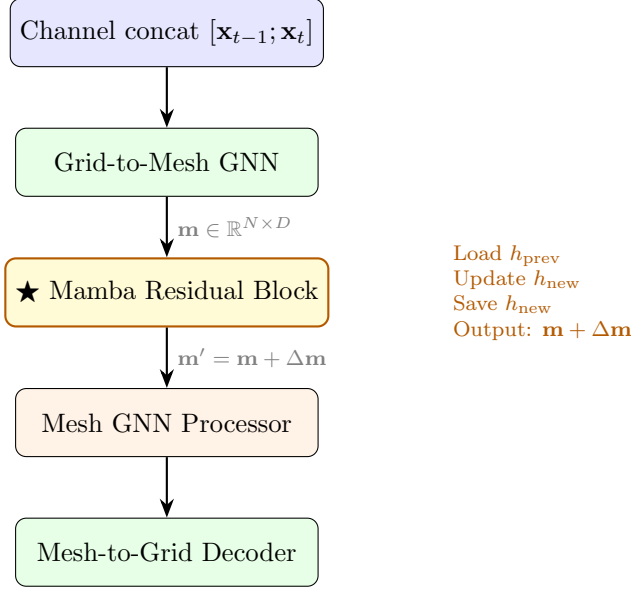


Figure 2: Mamba is inserted as a residual block between encoder and processor. Everything above and below it is identical to baseline GraphCast.

**Key design choice:** The encoding path (channel concatenation  $\rightarrow$  single Grid-to-Mesh pass) is **identical** to the baseline. Mamba only adds a residual correction. This ensures any performance difference is due to temporal memory, not a different encoding.

### 3.2 What Mamba Does at Each Prediction Step

During autoregressive rollout, Mamba maintains a hidden state  $h$  that carries information across prediction steps:

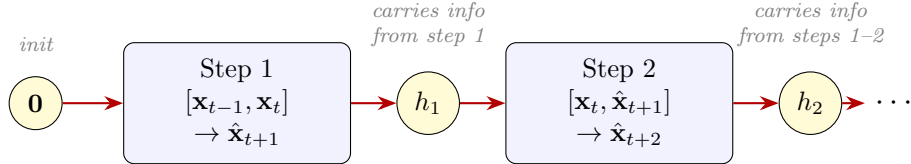


Figure 3: Hidden state  $h$  is passed from one prediction step to the next. At step  $k$ ,  $h_k$  contains a compressed summary of the forecast trajectory from the initial condition through step  $k$ .

At each step, Mamba:

1. **Loads** the hidden state  $h_{\text{prev}}$  from the previous step
2. **Reads** the current mesh latent  $\mathbf{m}$  (which encodes the current atmospheric state)
3. **Updates:**  $h_{\text{new}} = \text{decay} \cdot h_{\text{prev}} + (1 - \text{decay}) \cdot \text{proj}(\mathbf{m})$
4. **Outputs** a residual correction:  $\Delta \mathbf{m} = f(h_{\text{new}}, \mathbf{m})$
5. **Saves**  $h_{\text{new}}$  for the next step

The baseline model has no equivalent—each prediction step is completely independent.

### 3.3 State Shape and Batch Handling

The hidden state has shape  $(N_{\text{mesh}}, H) = (2562, 128)$ , stored **per mesh node** and **independent of batch size**. Each mesh node’s 128-dimensional state encodes temporal information about the atmospheric evolution at that spatial location.

During training with `batch_size > 1`, the state is tiled across the batch for the forward pass, then averaged back to  $(N_{\text{mesh}}, H)$  before saving. During inference with `batch_size = 1`, the state is exact.

## 4 Training: Sequential Sampling with Truncated BPTT

### 4.1 The Problem with Random Sampling

Standard training draws random time windows from the training data. Each sample is independent (i.i.d.), and the Mamba state is **reset to zero** before every sample. With `target_steps=2`, Mamba sees only 1 state transfer per sample—far too little to learn useful temporal patterns.

### 4.2 Our Solution: Chunked Sequential Sampling

We divide the training data into contiguous segments and iterate through them sequentially, carrying the Mamba state forward within each segment:

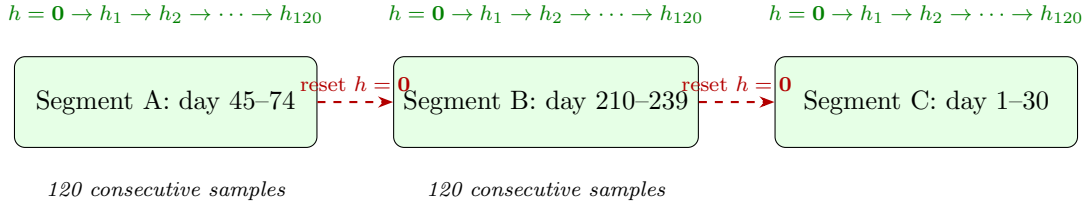


Figure 4: Segments are shuffled across epochs. Within each segment, data is sequential and state carries forward. State resets at segment boundaries.

### 4.3 Step-by-Step Training Procedure

1. **Segment construction:** Divide the 2020–2021 ERA5 data (2,920 time steps at 6h intervals) into contiguous segments of `sequential_segment_steps=120` steps (30 days). This yields  $\sim 24$  segments.
2. **Epoch start:** Randomly shuffle the order of segments.
3. **Segment start:** Reset Mamba state to  $\mathbf{0}$ .
4. **For each consecutive sample within the segment:**
  - (a) **State handling:**
    - First sample in segment:  $h = \mathbf{0}$  (cold start)
    - Otherwise:  $h = \text{stop\_gradient}(h_{\text{prev}})$  — state values are preserved but **gradients are detached**
  - (b) **Build training sample:** Extract consecutive window from the data:
    - Input:  $[\mathbf{x}_i, \mathbf{x}_{i+1}]$  (2 frames, 12 hours)
    - Targets:  $[\mathbf{x}_{i+2}, \dots, \mathbf{x}_{i+1+K}]$  ( $K = \text{target\_steps}$  future frames)
  - (c) **Forward pass:** Run  $K$  autoregressive rollout steps through GraphCast + Mamba. At each step, Mamba loads  $h$ , processes the mesh latent, saves updated  $h$ .
  - (d) **Loss:**  $\mathcal{L} = \frac{1}{K} \sum_{k=1}^K \text{wMSE}(\hat{\mathbf{x}}_{i+1+k}, \mathbf{x}_{i+1+k})$
  - (e) **Backward pass:** Compute gradients of  $\mathcal{L}$  w.r.t. all parameters. Gradients flow through the  $K$  rollout steps and through the Mamba state chain ( $h_K \rightarrow h_{K-1} \rightarrow \dots \rightarrow h_1$ ), but **stop at  $h_{\text{prev}}$**  due to `stop_gradient`.

- (f) **Parameter update:** Apply AdamW optimizer.
  - (g) **Save state:** Store  $h_K$  for the next sample.
5. **Segment end:** Move to next segment, reset  $h = \mathbf{0}$ .
  6. **Epoch end:** Reshuffle segments, repeat.

#### 4.4 Truncated BPTT: What Gradients Can and Cannot See

The `stop_gradient` at sample boundaries creates a “truncation window” of  $K$  steps:

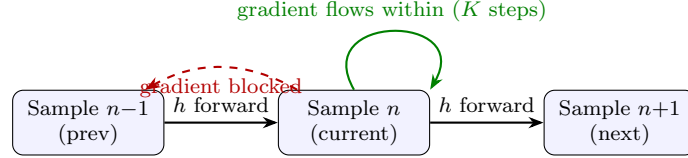


Figure 5: Forward: state flows across all samples in a segment. Backward: gradient only flows within the current sample’s  $K$  rollout steps.

##### What the model learns:

- **Reading state:** The model learns to extract useful information from  $h_{\text{prev}}$  (which contains accumulated history). This is trained because  $h_{\text{prev}}$  affects predictions within the current sample.
- **Writing state:** The model learns what to store in  $h$  that helps predictions  $1-K$  steps later. This is trained through the gradient chain within the sample.
- **Not trained:** How to write state that helps predictions  $> K$  steps later. The writing policy is optimized for short-term utility but may naturally produce state that is also useful long-term (due to the SSM decay dynamics).

## 5 Inference Procedure

At inference time, the model receives an initial condition and rolls out freely:

1. Initialize  $h = \mathbf{0}$
2. For each rollout step  $k = 1, 2, \dots, T$ :
  - (a) Encode input via channel concatenation + Grid-to-Mesh GNN  $\rightarrow$  mesh latent  $\mathbf{m}_k$
  - (b) Mamba: load  $h_{k-1}$ , update to  $h_k$ , compute residual  $\Delta\mathbf{m}_k$
  - (c) Add residual:  $\mathbf{m}'_k = \mathbf{m}_k + \Delta\mathbf{m}_k$
  - (d) Mesh GNN + decode  $\rightarrow$  prediction  $\hat{\mathbf{x}}_{t+6kh}$
  - (e) Feed prediction back as input for step  $k+1$

There is no `stop_gradient` during inference. After 2–3 steps,  $h$  has accumulated enough information to resemble the training-time state (which was built up over many samples within a segment). This is when Mamba’s temporal memory begins to help.

The inference `target_steps`  $T$  can be **different from training**. We train with  $K=2$  (12h) but evaluate at  $T=2, 10, 16, 24$  (up to 6 days).

## 6 Complete Parameter Table

Table 1: All parameters with physical meaning. Bold values indicate the current default.

| Parameter              | Value            | Physical Meaning                                                                                               |
|------------------------|------------------|----------------------------------------------------------------------------------------------------------------|
| <i>GraphCast model</i> |                  |                                                                                                                |
| resolution             | <b>2.0°</b>      | Spatial grid spacing ( $\sim 200$ km). Also tested: 4.0°, 6.0°.                                                |
| mesh_size              | <b>4</b>         | Icosahedral refinement level. $4 \rightarrow 2,562$ mesh nodes on the sphere.                                  |
| width (latent_size)    | <b>128</b>       | Feature dimension at each node. Controls model capacity.                                                       |
| gnn_msg_steps          | <b>1</b>         | Message-passing rounds in mesh processor. More = longer-range spatial communication.                           |
| input_duration         | <b>12h</b>       | Input time window: 12h = 2 frames at 6h intervals.                                                             |
| pressure_levels        | <b>13</b>        | Number of vertical levels (50–1000 hPa).                                                                       |
| <i>Mamba SSM</i>       |                  |                                                                                                                |
| hidden_size $H$        | <b>128</b>       | State dimension per mesh node. Memory capacity.                                                                |
| layers                 | <b>1</b>         | Number of stacked SSM blocks.                                                                                  |
| dropout                | <b>0.0</b>       | Dropout on SSM output.                                                                                         |
| a_log init             | <b>−0.1</b>      | Log-decay initialization. $-0.1 \rightarrow \text{decay} \approx 0.4/\text{step}$ . Lower = faster forgetting. |
| skip init              | <b>1.0</b>       | Skip connection weight at initialization.                                                                      |
| location               | <b>residual</b>  | Insertion point: after encoder, before processor. Same encoding as baseline.                                   |
| <i>Training</i>        |                  |                                                                                                                |
| target_steps $K$       | <b>2</b>         | Rollout steps per training sample. Also tested: 4, 6, 8.                                                       |
| segment_steps          | <b>120</b>       | Sequential segment length (120 steps = 30 days).                                                               |
| batch_size             | <b>1</b>         | Samples per gradient update.                                                                                   |
| max_steps              | <b>10,000</b>    | Total training iterations.                                                                                     |
| lr                     | $10^{-4}$        | AdamW learning rate.                                                                                           |
| weight_decay           | $10^{-4}$        | L2 regularization.                                                                                             |
| precision              | <b>bf16</b>      | Floating-point format.                                                                                         |
| train_years            | <b>2020–2021</b> | Training data (ERA5, 6h intervals).                                                                            |
| <i>Inference</i>       |                  |                                                                                                                |
| Inference $T$          | <b>2–24</b>      | Rollout steps at eval time. Independent of training $K$ .                                                      |
| eval_year              | <b>2022</b>      | Held-out evaluation year.                                                                                      |



## 7 Preliminary Results

Table 2: Long rollout inference: baseline vs. sequential resmem (both trained with  $K=2$ , evaluated at step 10k). Results obtained with the `--max-steps 1` workaround; corrected `--eval-only` results pending.

| Inference $T$ | Forecast | Baseline RMSE | Resmem Seq RMSE | Improvement |
|---------------|----------|---------------|-----------------|-------------|
| 2             | 12h      | 120.35        | <b>87.04</b>    | −27.7%      |
| 10            | 2.5 days | 689.56        | <b>351.93</b>   | −49.0%      |
| 16            | 4 days   | 881.08        | <b>756.74</b>   | −14.1%      |
| 24            | 6 days   | 848.69        | <b>623.27</b>   | −26.6%      |

The sequential resmem model shows 14–49% RMSE improvement on long rollout inference. The paradoxical result: on standard training evaluation (state=zero,  $K=2$ ), resmem is 7% *worse* than baseline. This confirms Mamba learned to depend on its hidden state—when state is available (long rollouts), it helps dramatically; when removed (training eval), the model underperforms.

Corrected evaluation results using `--eval-only` mode and experiments with varying training `target_steps` (4, 6, 8) and spatial resolution ( $4^\circ$ ,  $6^\circ$ ) are in progress.

## References

- [1] R. Lam et al., “Learning skillful medium-range global weather forecasting,” *Science*, vol. 382, no. 6677, pp. 1416–1421, 2023.
- [2] A. Gu and T. Dao, “Mamba: Linear-time sequence modeling with selective state spaces,” *arXiv preprint arXiv:2312.00752*, 2023.